WK 11: What Changes at Scale

Spark Under the Hood · Algorithmic Thinking · The Limits of Exactness

Phase 3: Distributed Computing

THE DISTRIBUTED COMPUTING ARC

✓ — ✓ — 11 — 12

Week 9
Concepts: MapReduce, fault tolerance

Week 10
Managed Platforms: Spark advantages, Databricks

Week 11
Under the Hood: Optimization, algorithm choice

Week 12
Hands-On: Local PySpark, EMR cluster

Last week you ran queries in Databricks. It was fast. But why?

This week: how it works and why it matters

TODAY'S ARGUMENT IN ONE SENTENCE

"As data grows, everything must change: the execution engine, the algorithm, and eventually the assumption that the data has an endpoint."

1

Act 1: What Spark Does for You

The engine defers decisions and optimizes globally. The user been benefits without knowing it.

2

Act 2: What You Need to Do Differently

The engine can't choose a different algorithm. As data grows larger, the algorithm itself must change.

3

Act 3: What Happens When Data Never Stops

Streaming is the limit case: when data are unbounded and speed matters most, every optimization is vital.

WHAT HAPPENED WHEN YOU HIT "RUN"?

What You Saw

Type a query in a Databricks notebook.

Press Run.

Results appear.

What Actually Happened

Spark analyzed the entire query, rearranged operations, chose strategies, compiled optimized code, distributed work across machines, then began computation.

Think of a GPS: if you gave it one turn at a time, it couldn't find a shortcut. Spark needs the complete destination before it plans the route.

Spark is a planner and optimizer, not a line-by-line executor.

LAZY EVALUATION: BUILD A PLAN, DON'T EXECUTE YET

Transformations — Build the Plan

No execution, builds the plan.

· filter rows
· select columns
· join tables
· group by a key

Instant return = no data processed

Actions — Trigger Execution

Triggers real computation

· count rows
· show results
· write output

Nothing happens until one of these fires

Spark needs the complete plan before it can optimize. If you filter after a join, Spark may rearrange to filter before the join — but only if it saw both instructions together.

THE DAG: SPARK'S EXECUTION BLUEPRINT

DAG = Directed Acyclic Graph

Directed

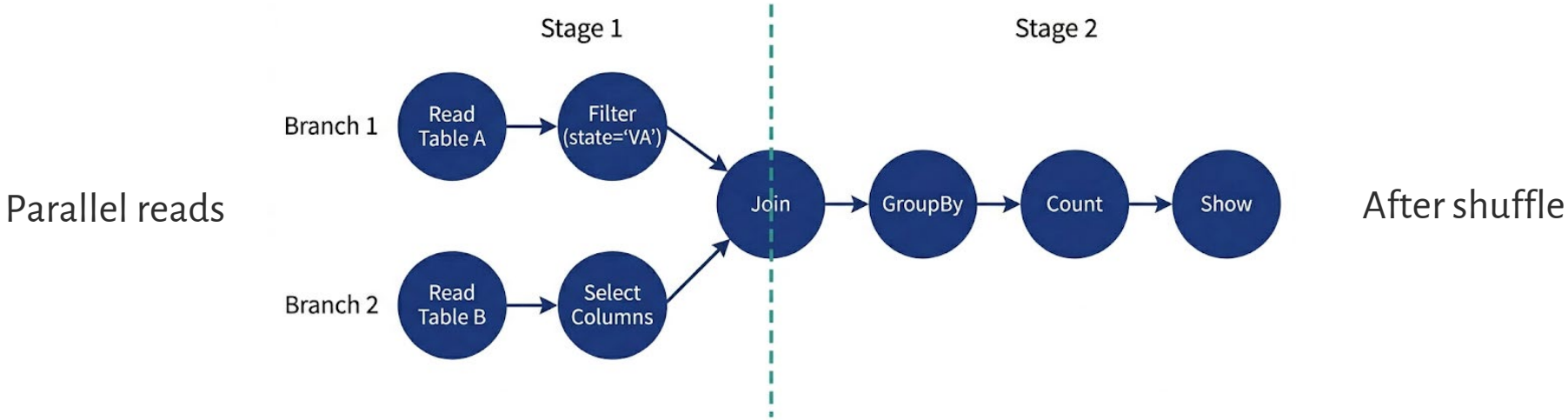
Data flows one direction (input → output)

Acyclic

No loops (data never flows backward)

Graph

A network of stages connected by dependencies



Week 9: MapReduce = rigid two-phase (Map → Reduce). The DAG generalizes this: Spark can have any number of stages with complex dependencies.

NARROW VS. WIDE TRANSFORMATIONS

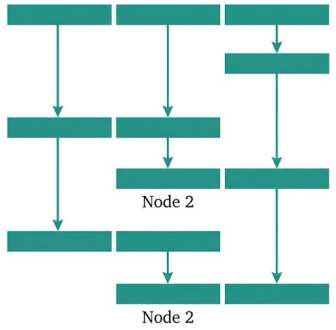
Narrow: Data Stays Put

- Each output partition depends on exactly one input partition.
- No network transfer.
  - filter, select
  - withColumn, union
- Each worker processes independently.

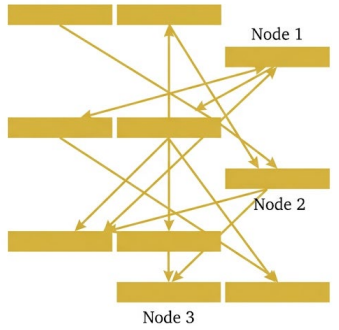
Wide: Data Must Move

- Each output partition depends on multiple input partitions.
- Requires network transfer (shuffle).
  - groupBy, join
  - distinct, orderBy
- Workers must exchange data.

Narrow = cheap (pipeline in memory). Wide = expensive (triggers a shuffle).

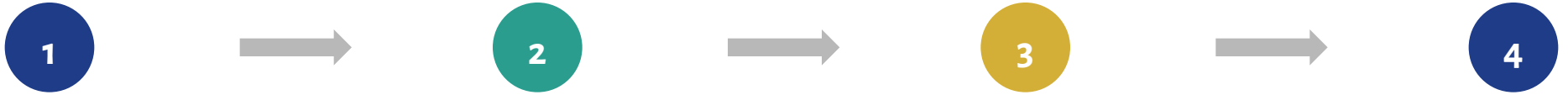


Narrow — Data stays local.



Wide — Data moves across network.

THE SHUFFLE: WHY WIDE TRANSFORMATIONS ARE EXPENSIVE



Serialize
Convert to bytes

Write Disk
Local disk writes

Transfer
Network (all-to-all)

Deserialize
Read + merge

Shuffles are expensive because they involve all three bottlenecks simultaneously: CPU (serialization), disk I/O (local writes/reads), and network (data transfer).

Within a Stage

RAM ↔ RAM
Pipelines in memory (nanoseconds)

Across a Shuffle

Disk → Network → Disk
Milliseconds to seconds

Reducing shuffles is the single highest-leverage optimization in Spark.

SPARK'S THREE LAYERS OF OPTIMIZATION

What does Spark handle for the user? Three layers work on their behalf:

Catalyst Optimizer

- Rewrites your logical plan for efficiency.
- Before execution.

Adaptive Query Engine (AQE)

- Adjusts the plan at runtime based on actual data sizes and execution statistics.
- Re-optimizes after each shuffle stage.
- During execution.

Bloom Filter Joins

- Applies probabilistic filtering to reduce shuffle cost.
- Within execution.

Understanding what each layer does — and where they stop — is how to reason about performance.

LAYER 1: THE CATALYST OPTIMIZER

Spark's query optimizer — rewrites your query plan before execution. Same idea as SQL query planning (Week 5).

Predicate Pushdown

You write: join → filter.
Catalyst rearranges: filter → join.
Less data to shuffle.

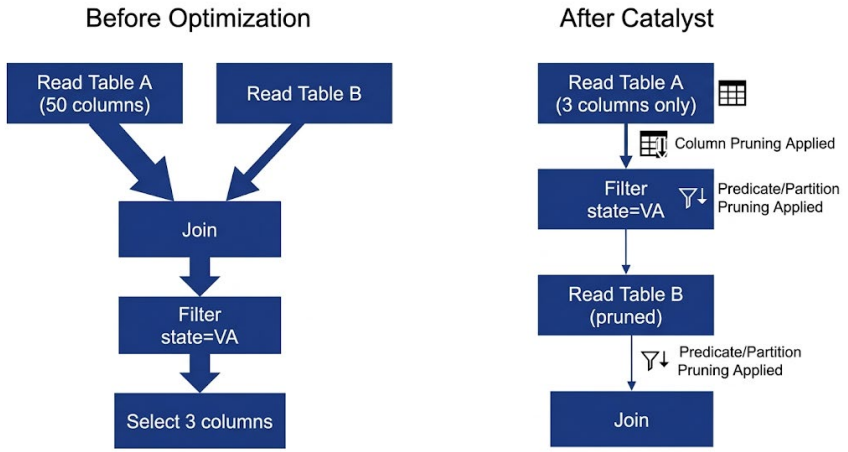
Column Pruning

Output uses 3 of 50 columns.
Catalyst skips reading the other 47 from Parquet.

Join Reordering

For multi-table joins, Catalyst picks the order producing the smallest intermediate results.

df.join(other).filter(condition) and df.filter(condition).join(other) may produce the same physical plan.



LAYER 2: ADAPTIVE QUERY EXECUTION (AQE)

Runtime optimization that adjusts the plan while running, based on actual data sizes. Default since Spark 3.2 (2022).

Coalesces Partitions

After a shuffle, some partitions are tiny. AQE merges them automatically.

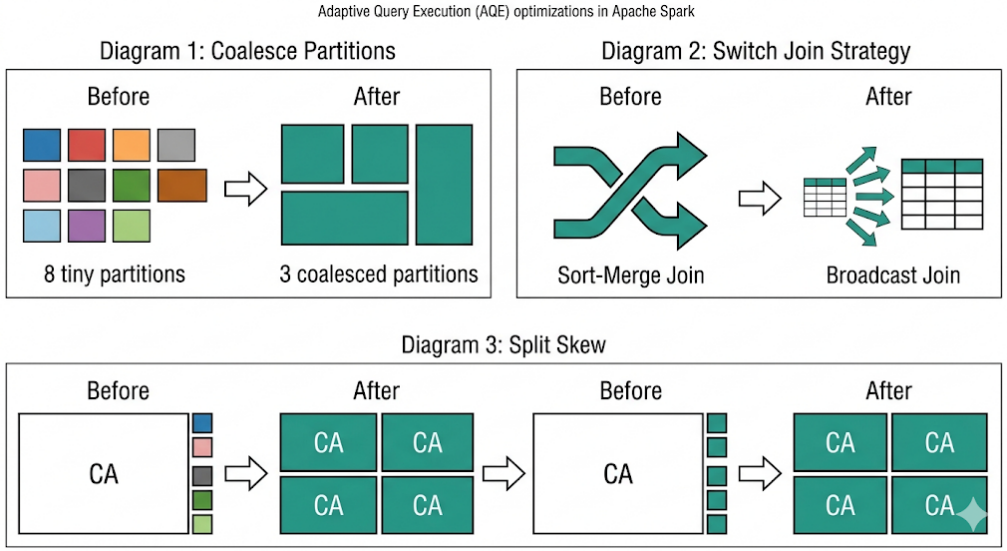
Switches Join Strategy

If a filter reduces 10GB to 50MB, AQE switches from Sort-Merge to Broadcast Join mid-execution.

Splits Skewed Partitions

90% of records in California, 1% in Wyoming? AQE detects the straggler and splits it.

Manual partition tuning is largely obsolete in 2026 for standard batch workloads. Trust AQE for partitions and joins.



WHAT AQE CANNOT FIX

Bad Data Layouts

Extreme skew or poor storage format that no partition splitting can resolve.

An Unbounded Shuffle

If the query requires moving the entire dataset across the network, no adjustment makes that cheap.

A Wrong Algorithm

AQE can choose the best join strategy. It cannot choose a different algorithm for your computation.

Layer	What	When	Your Role
Catalyst	Rewrites logical plan	Before execution	Write clear queries
AQE	Adjusts physical plan	During execution	Trust it for partitions
You	**Choose the algorithm**	**Before writing query**	**This is Act 2**

The optimizer stops here. Algorithm choice is your job. → Act 2

LAYER 3: BLOOM FILTER JOINS

A probabilistic data structure: "Is this element in the set?"

How It Works

Bit array (0s and 1s).
Hash element → set positions.
Query: check positions
- All 1s → probably in set.
- Any 0 → definitely not.

Key Properties

No false negatives.
Small false positive rate.
Very compact: far less memory than storing actual set.
Named after Burton Bloom (1970).

How Spark Uses It

Before shuffle for a join:
- Build filter from small table.
- Check each large table row.
- Drop definite non-matches.
- Massive shuffle reduction.

You've been using approximate algorithms without knowing it. This is the same paradigm shift we'll study in Act 2.

THE ENGINE UNDER THE HOOD: TUNGSTEN

Spark's execution engine: Tungsten processes data after the optimizer builds the plan.

Manual Memory Mgmt

Instead of JVM garbage collection, Tungsten manages memory directly using compact byte arrays. Avoids GC pauses that stalled executors.

Code Generation

Instead of interpreting one operator at a time, Tungsten generates optimized bytecode for entire pipeline stages (one tight loop).

2015
Catalyst + Tungsten

2020
AQE introduced

2022
AQE default

2025
Spark 4.0

Plan smarter (Catalyst) → Execute faster (Tungsten) → Adapt at runtime (AQE)

WHY DATAFRAMES ARE FAST (AND WHY PYSPARK ISN'T SLOW)

RDDs (Opaque)

Optimizer can't see Python functions.
No predicate pushdown.
No column pruning.
No code generation.
Data serialized JVM ↔ Python.

DataFrames (Transparent)

Operations express intent.
Spark sees logical meaning.
Full optimization applies.
PySpark = same speed as Scala.

Use Python as the control plane for building plans, not the data plane for processing rows.

Efficient Spark Code

```
# ── TIER 1: Spark Built-in Functions
(fastest) ──────────
# Imported from pyspark.sql.functions →
optimizer sees everything

from pyspark.sql.functions import col,
regexp_replace, when, lit

df.withColumn("clean_phone",
regexp_replace(col("phone"), "
[()\\s-]", "")) \
  .withColumn("tier",
when(col("visits") <= 2, "New")

.when(col("visits") <= 10, "Regular")

.otherwise("VIP"))
```

```
# ── TIER 2: Pandas UDF (medium)
──────────────────────────

# Processes column batches via Arrow —
black box to optimizer

@pandas_udf(StringType())
def clean_phone(phones: pd.Series) ->
pd.Series:
    return phones.str.replace(r"
[()\\s-]", "", regex=True)


# ── TIER 3: Python UDF (slowest)
──────────────────────────

# Serializes every row JVM → Python →
JVM — black box to optimizer

@udf(returnType=StringType())
def clean_phone(phone):
    return phone.replace("
(","").replace(")","").replace("-","")
```

WHAT DATABRICKS ADDS: PHOTON

Last week you ran Photon, not open-source Tungsten. A Databricks-proprietary C++ engine.

	Tungsten (open-source)	Photon (Databricks)
Language	JVM (Java bytecode)	Native C++
Processing	Row-at-a-time within fused stages	Column batches (vectorized)
CPU Optimization	Code generation	SIMD instructions
Cost	Open-source	Premium DBUs

Photon is most beneficial for heavy aggregations and analytics. For simple data movement, the premium cost may not

Understanding the API vs. the engine underneath matters for cost and performance decisions.

SPARK CONNECT: WHAT HAPPENS WHEN YOU RUN A CELL

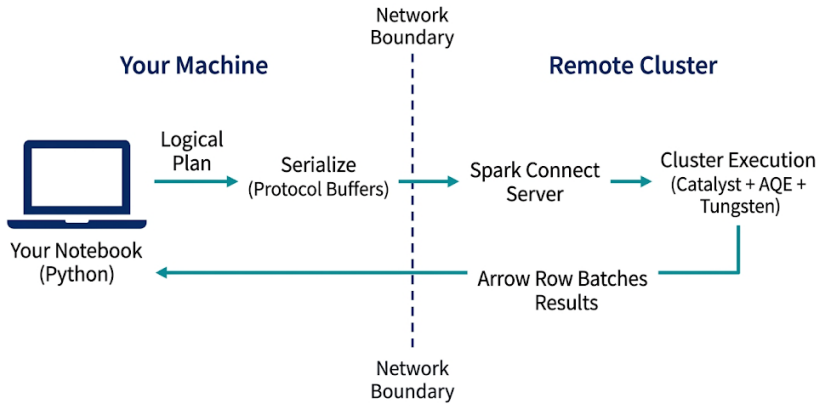
1. Your notebook builds a query plan (Python)
2. Plan serialized (Protocol Buffers)
3. Sent over network (gRPC)

Your Machine

4. Spark server: Catalyst + AQE + Tungsten/Photon
5. Results stream back as Arrow batches
6. Notebook displays them

Remote Cluster

Your notebook is a thin client. It never touches the data. The plan crosses a network boundary; execution happens on



DEPLOYMENT: KUBERNETES AND SERVERLESS SPARK

	YARN (2010s)	Kubernetes (2020s)
Designed for	Hadoop ecosystem only	Any containerized software
Scaling	Static, manual sizing	Dynamic, auto-scales
Infrastructure	Dedicated long-running	Ephemeral containers
Cloud	Limited integration	Native to all major clouds

YARN

You manage everything

Kubernetes

You manage containers

Serverless

You submit code

Decreasing operational burden →

ACT 1 SUMMARY: "DEFER AND OPTIMIZE"

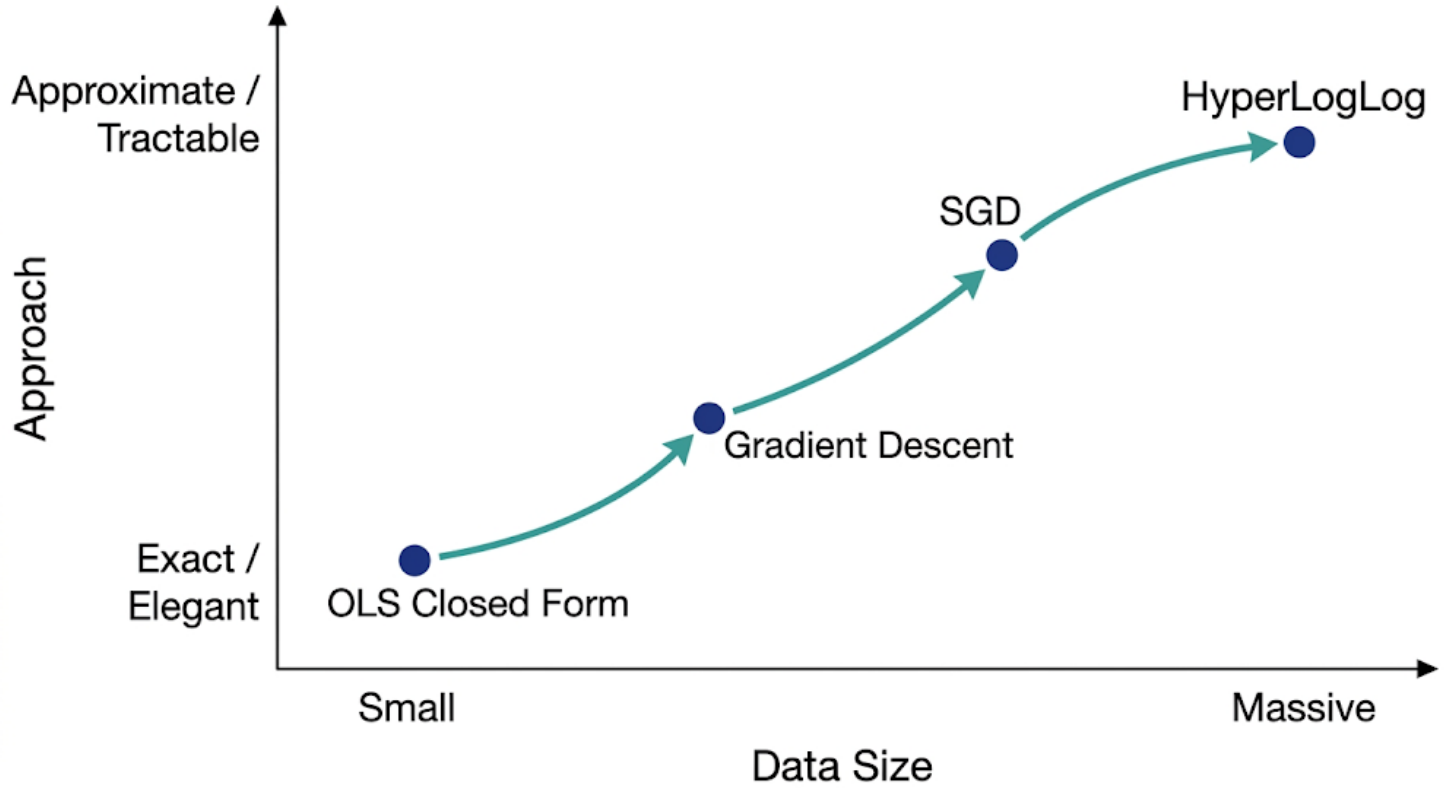
Everything in Act 1 follows one pattern repeated at every level:

Lazy evaluation:	defer execution until an action triggers it.
The DAG:	defer physical planning until the full plan is known.
Catalyst:	defer column reads and filter placement for optimizer rearrangement.
AQE:	defer final physical decisions until runtime statistics are available.
Bloom filter joins:	defer exact filtering in favor of probabilistic pre-filtering.

"But there's one thing none of these can defer: the choice of algorithm. If the algorithm doesn't scale, no amount of optimization will help.

THE ALGORITHM IS THE ARCHITECTURE

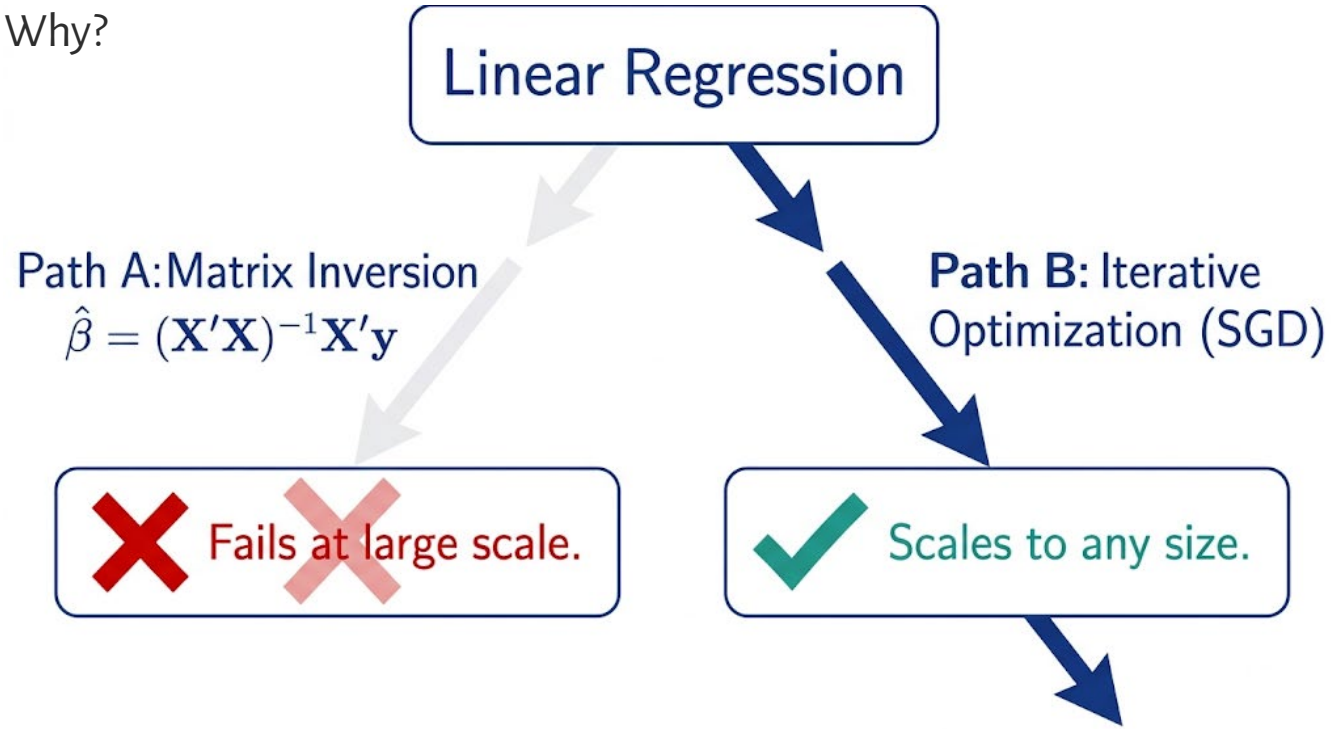
"Algorithm choice is an infrastructure decision." At small data, choose the elegant solution. At massive data, choose the tractable



Before scaling infrastructure, ask whether a different algorithm would solve the problem without the bigger cluster.

"WHICH LINEAR REGRESSION ALGORITHM?"

In Databricks MLlib, linear regression uses L-BFGS or mini-batch SGD — not the ($\beta = (X'X)^{-1}X'y$) formula. Same model. Different algorithm. Why?



The same model can be computed by fundamentally different algorithms. At scale, which algorithm you choose matters more than how many machines you have.

OLS CLOSED FORM: THE HIDDEN COMPUTATIONAL COST

$\beta = (X'X)^{-1}X'y$ **Matrix inversion is $O(p^3)$. Memory for $X'X$ is p^2 floats.**

Features (p)	Inversion Time	Memory for X'X
100	Trivial	80 KB
10,000	~5 minutes	800 MB
100,000	**~3.5 days**	**80 GB**

statsmodels.OLS(...).fit() defaults to matrix decomposition (similar cost).
sklearn.SGDRegressor uses stochastic gradient descent, a fundamentally different algorithm.

The formula from econometrics class has a hidden computational cost that only matters at scale.

GRADIENT DESCENT: REPLACING ALGEBRA WITH ITERATION

1. Start

Initialize parameters at random starting point.

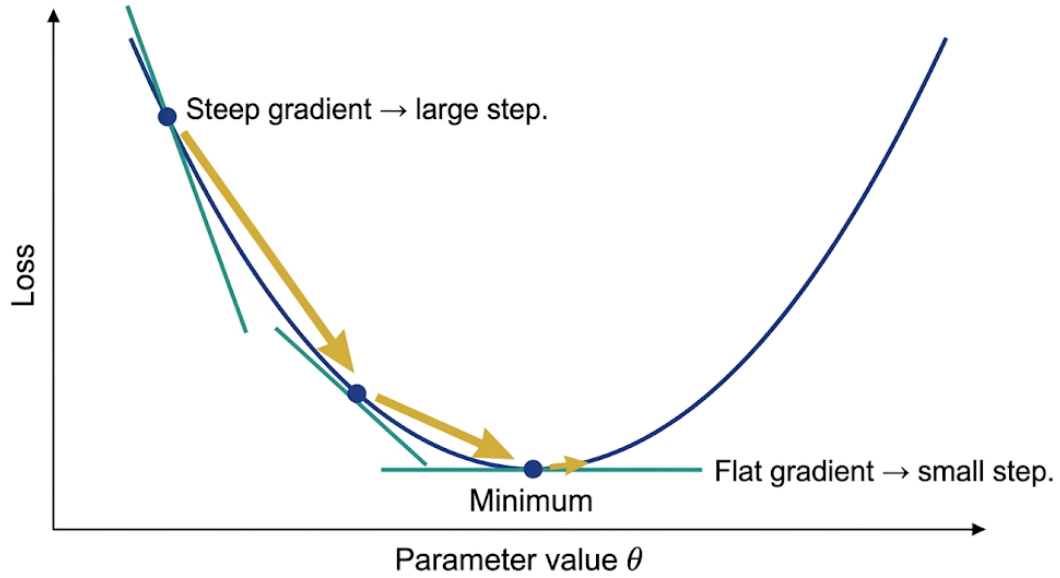
2. Compute Gradient

Calculate slope of loss surface at current position.

3. Step Downhill

Move parameters in the direction that reduces loss.

Repeat steps 2-3 until convergence.
Cost: O(np) per step.
Still needs all data in memory.



SGD AND MINI-BATCH: THE WORKHORSES

Batch GD

One large step per epoch.
Uses all data.
Cost: O(np) per step.
Smooth but slow.

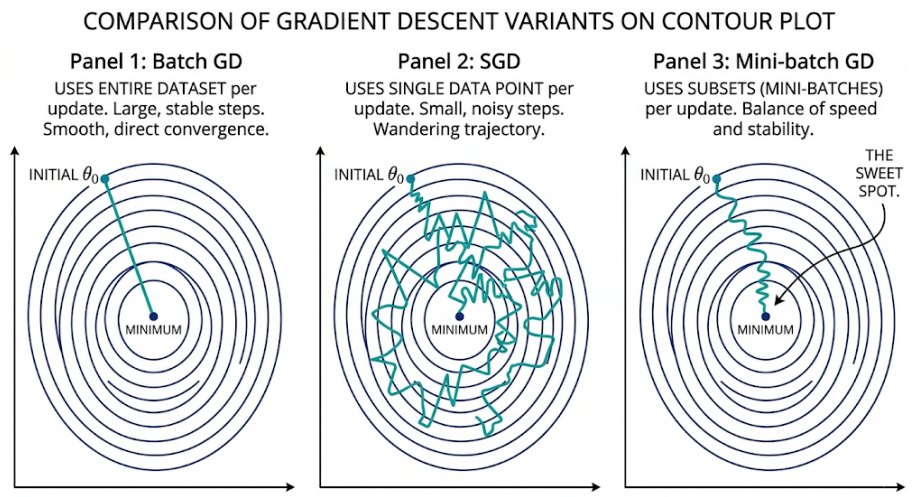
SGD

Many small noisy steps.
One sample at a time.
Cost: O(p) per step.
Fast but noisy.

Mini-Batch

The sweet spot.
Batch of k samples.
Cost: O(kp) per step.
Naturally parallelizable.

Deeper principle: substituting statistical approximation for algebraic precision. The noise is sometimes beneficial (implicit regularization).



MLE: ONE FRAMEWORK, MANY MODELS

Maximum Likelihood Estimation reframes any parametric model as an optimization problem.
Once you see this, SGD works for everything with a differentiable likelihood.

Logistic Regression

Binary outcomes.
Log-likelihood.
SGD works.

Poisson Regression

Count data.
Log-likelihood.
SGD works.

Neural Networks

Complex functions.
Cross-entropy loss.
SGD is standard.

All feed into the same optimization machinery. MLE unifies model specification and computation.

JAX AND AUTOMATIC DIFFERENTIATION

Modern tools compute exact derivatives automatically from code.
JAX replaces manual calculus with compiler-level differentiation.

You write the loss function.

System computes the gradient.

No manual calculus required.

Numerical Approximation (Finite Differences)

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Approximate — accuracy depends on h.

$$f(x) = \sin(x), h = 0.001,$$
$$f'(1.0) \approx 0.5403\ldots$$

Automatic Differentiation (JAX)

$$\text{grad}(\sin)(x) \longrightarrow \cos(x)$$

Exact — computed via chain rule, not approximation.

$$\text{grad}(\sin)(1.0) = \cos(1.0)$$
$$= 0.54030230586\ldots$$

JAX applies the chain rule automatically to any Python function — no manual calculus, no approximation error.

JAX, PyTorch autograd, and TensorFlow GradientTape all do this. The manual calculus bottleneck is gone.

THE CURRENT OPTIMIZER LANDSCAPE

2014

Adam

Adaptive learning rates

2019

AdamW

Fixes weight decay

2023

Lion

Halves memory

2023

Sophia

Uses curvature info

Green AI argument: efficiency is an equity issue. Optimizer choice determines compute cost, energy use, and who can afford to train models.

Optimizer choice is an active research frontier. The defaults have shifted since older textbooks.

BIG-O: GIVING COMPLEXITY A NAME

Complexity	Name	Visceral Example
O(n)	Linear	1 sec at n=1K → 1,000 sec at n=1M
O(n log n)	Linearithmic	Sorting (practical ceiling for most operations)
$O(n^2)$	**Quadratic**	**1 sec at n=1K → 11.5 days at n=1M**
$O(n^3)$	Cubic	Matrix inversion (the OLS wall)

$O(n^2)$: 1 second at n=1,000 → 11.5 days at n=1,000,000
This isn't "a bit slower." This is the difference between finishing before lunch and running for two weeks.

Before scaling infrastructure, check the algorithm's complexity class.

CHANGING THE ALGORITHM WITHOUT CHANGING THE PLATFORM

Modin

Drop-in pandas replacement.
Parallelizes across cores.
Same API, faster.

Dask

Parallel larger-than-memory DataFrames.
Task graph like a DAG.
Lazy evaluation.

Ray

General distributed execution.
Can parallelize sklearn.
Flexible framework.

pandas (<1GB) → Modin/Dask/Ray (1GB–100GB) → Spark (100GB+)

The journey to Spark has intermediate steps. These tools change the execution strategy under the hood: same code, different computational approach.

THE ALGORITHM-ARCHITECTURE FLIP

Resource	Single Machine	Distributed Cluster
Compute	Abundant (one fast CPU)	Abundant (many CPUs)
Memory	Limited (GB scale)	Abundant (TB aggregate)
Communication	**Free (shared memory)**	**EXPENSIVE (network)**
Iterations	**Cheap (no coordination)**	**Expensive (synchronization)**

The scarce resource flips when you move from one machine to a cluster. Algorithms optimized for one environment may be actively harmful in the other.

THE FLIP: OLS EXAMPLE

OLS: Single Machine

One step, exact result.
Serial $O(p^3)$ bottleneck.
Works if p is small.
No communication needed.

OLS: Cluster

Distributed X'X aggregation is cheap.
BUT inversion is still serial $O(p^3)$.
Cluster doesn't help the bottleneck.

SGD: Single Machine

Many iterations needed.
Wasteful if data fits in memory.
But works when p is huge.

SGD: Cluster

Mini-batches distribute naturally.
Gradient aggregation is cheap.
Scales with both n and p.

SGD's communication cost is $O(p)$ per step. OLS inversion is $O(p^3)$ regardless of cluster size.

MORE EXAMPLES OF THE FLIP

Problem	Single Machine	Distributed
Sorting	Quicksort O(n log n)	External merge sort (shuffle)
Distinct count	HashSet (exact)	HyperLogLog (approximate)
k-NN	KD-tree O(n log n)	LSH approximate neighbors
Decision trees	Full tree in memory	Distributed histogram splits

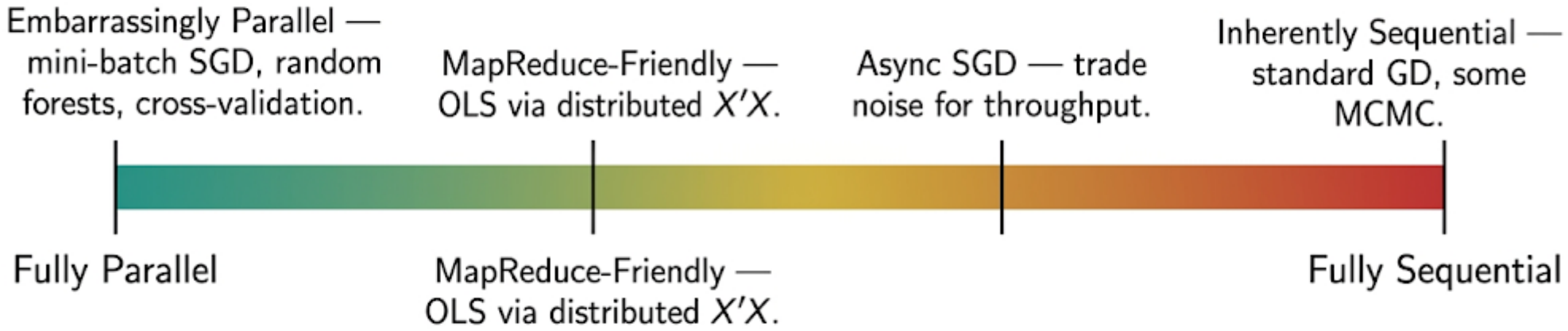
Each distributed version uses a different algorithm — often an approximation. The "right" algorithm depends on where you're computing.

THREE-ENVIRONMENT MENTAL MODEL

	CPU	Cluster	GPU
Scarce resource	Operations	Communication	Memory bandwidth
Optimize for	Minimize FLOPs	Minimize shuffles	Maximize arithmetic intensity
Algorithm style	Exact, algebraic	Communication-avoiding	Vectorized, batched

Same algorithm, three environments, three different bottlenecks. Where you compute determines which algorithm is optimal.

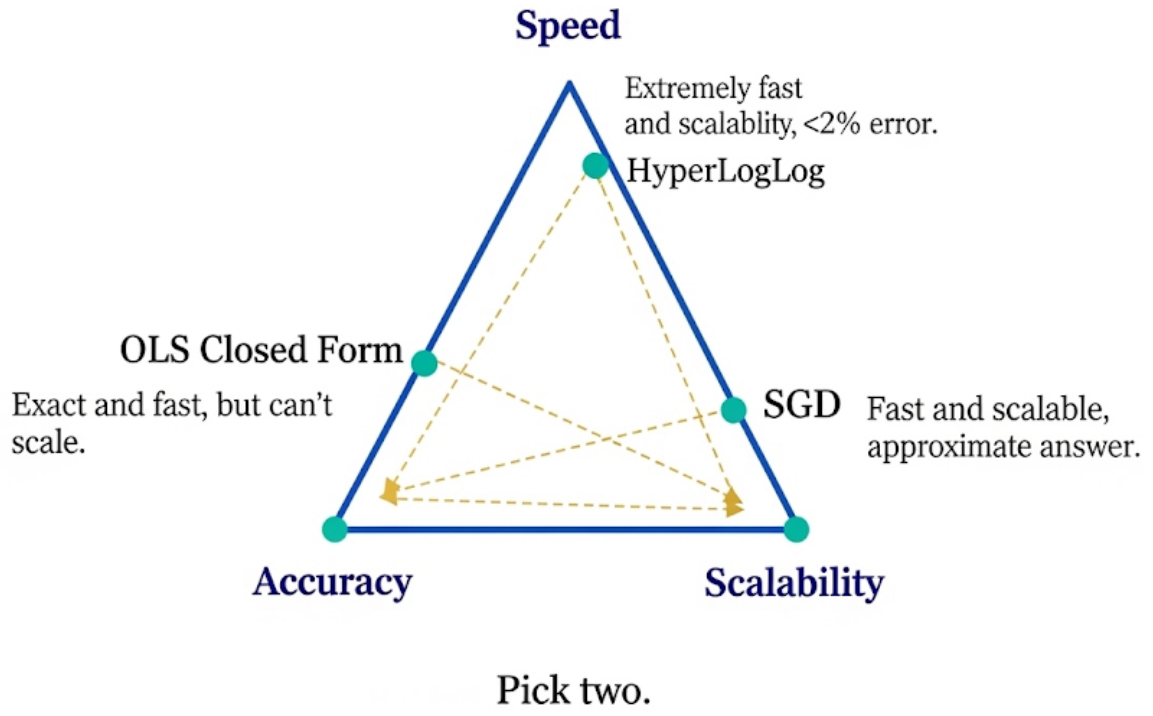
PARALLELIZABLE VS. SEQUENTIAL



Not every algorithm parallelizes equally. The degree of inherent parallelism is a key factor in choosing algorithms for distributed execution.

THE SPEED / ACCURACY / SCALABILITY TRIANGLE

Three vertices: two of three achievable at any given time.



Every algorithm makes a tradeoff. Understanding which vertex you're sacrificing is the key to informed algorithm selection.

THE PARADIGM SHIFT: FROM EXACT TO CLOSE ENOUGH

For policy students: what's the minimum accuracy needed for the decision at hand?

Exact

Precise answer.
Full computation.
May be infeasible at scale.
Required for audits, legal.

Approximate

Close enough.
Bounded error.
Massively scalable.
Sufficient for dashboards.

The Tradeoff

Error rate is tunable.
More memory = less error.
Always ask: how precise does this need to be?

The shift from exact to approximate is the defining paradigm change when working with massive data.

The question changes from How do I get the exact answer?
to What accuracy does this decision require?

HYPERLOGLOG: COUNTING THE UNCOUNTABLE

How many distinct users visited your site? At 1B events, exact counting needs ~8GB. HLL needs 12KB with ~2% error.

Intuition

Hash each element.
Count leading zeros.
More leading zeros = rarer pattern = larger set.
Like flipping coins.

The Tradeoff

12KB memory.
~2% standard error.
Count billions of distincts.
Native in Spark, BigQuery, Redis, Snowflake.

How It Scales

Memory: O(1) fixed.
Processing: one pass.
Mergeable across partitions.
Perfect for distributed.

12KB to count billions. This is what "approximate" buys you.

BLOOM FILTERS, COUNT-MIN SKETCH, T-DIGEST

Bloom Filter

Answers: "Is X in the set?"
No false negatives.
Small false positive rate.
Callback: Slide 13 —
Spark already uses these.

Count-Min Sketch

Answers: "How many times did X appear?"
Over-estimates, never under.
Used in network monitoring, frequency tracking.

T-Digest

Answers: "What's the 95th percentile?"
Accurate at extremes.
Mergeable across nodes.
Used in monitoring tools.

Each trades memory and exactness for massive scalability. Know which question each answers.

WHEN APPROXIMATION IS / IS NOT ACCEPTABLE

Acceptable ✓

Dashboards and monitoring
Exploratory analysis
Trend detection
Capacity planning
A/B test sizing

NOT Acceptable ✗

Financial audits
Election vote counting
Legal compliance reporting
Payroll calculations
Regulatory submissions

The question is never "is approximation OK?" — it's "what's the minimum precision this decision requires?"

SAMPLING: THE OLDEST APPROXIMATE ALGORITHM

When It Works

Population means.
Regression coefficients.
Precision: 1/√n.
10K samples often sufficient.

When It Breaks

Rare events (fraud, disease).
Tail distributions.
Small subgroups.
Sampling may miss them entirely.

Reservoir Sampling

Sample from a stream without knowing total size.
Keep k items, replace with decreasing probability.
One pass, fixed memory.

Precision scales as 1/√n — diminishing returns. 10K samples gives ~1% error for population means.

GOVERNMENT CASE STUDIES

IRS

$10.59B budget.
2.35 PB of data.
Approximate matching for fraud detection.
Exact for tax returns.

Census Bureau

Differential privacy in 2020 Census.
Adds calibrated noise to protect individuals.
Tradeoff: privacy vs. accuracy for small areas.

Even government agencies that require legal precision use approximation where the decision context permits it.

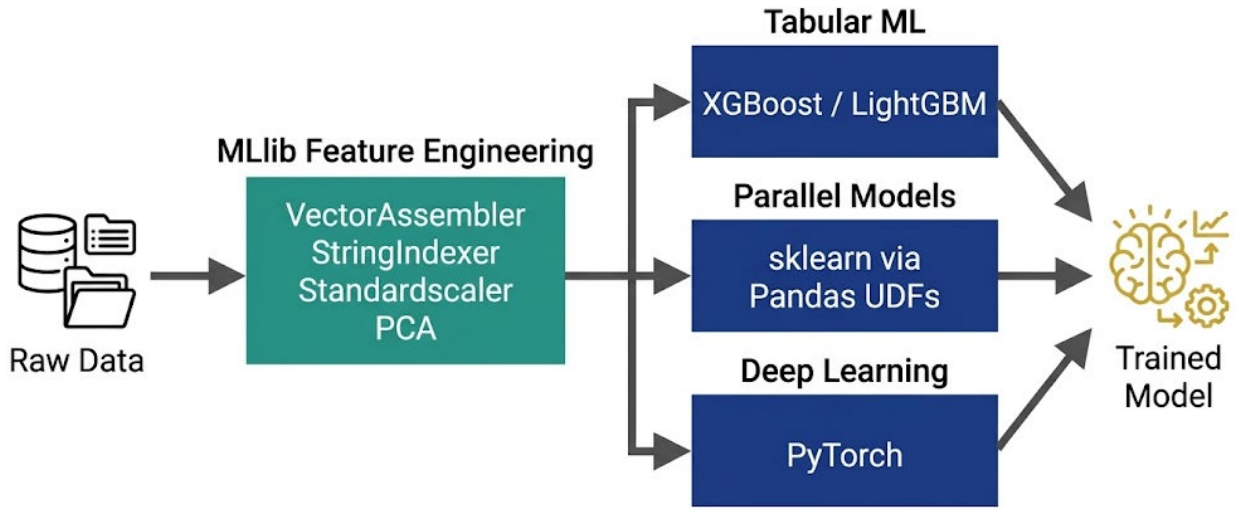
THE FIVE DIAGNOSTIC QUESTIONS

1. What's the complexity class of my algorithm?
2. Does it parallelize naturally, or is it inherently sequential?
3. What precision does this decision actually require?
4. Am I processing bounded or unbounded data?
5. Does my algorithm match my execution environment?

These five questions synthesize the entire distributed computing unit.
This is the portable framework you take with you.

MLLIB IN 2026: FEATURE ENGINEERING, NOT FULL TRAINING

MLlib is best used for feature engineering pipelines, not as a complete training library.



Pipeline API pattern: Data → MLlib feature transforms → Specialized training library → Model

MLlib handles the distributed data wrangling. Specialized libraries handle the training. Use each where it's strongest.

ACT 2 SUMMARY: THE PROGRESSION

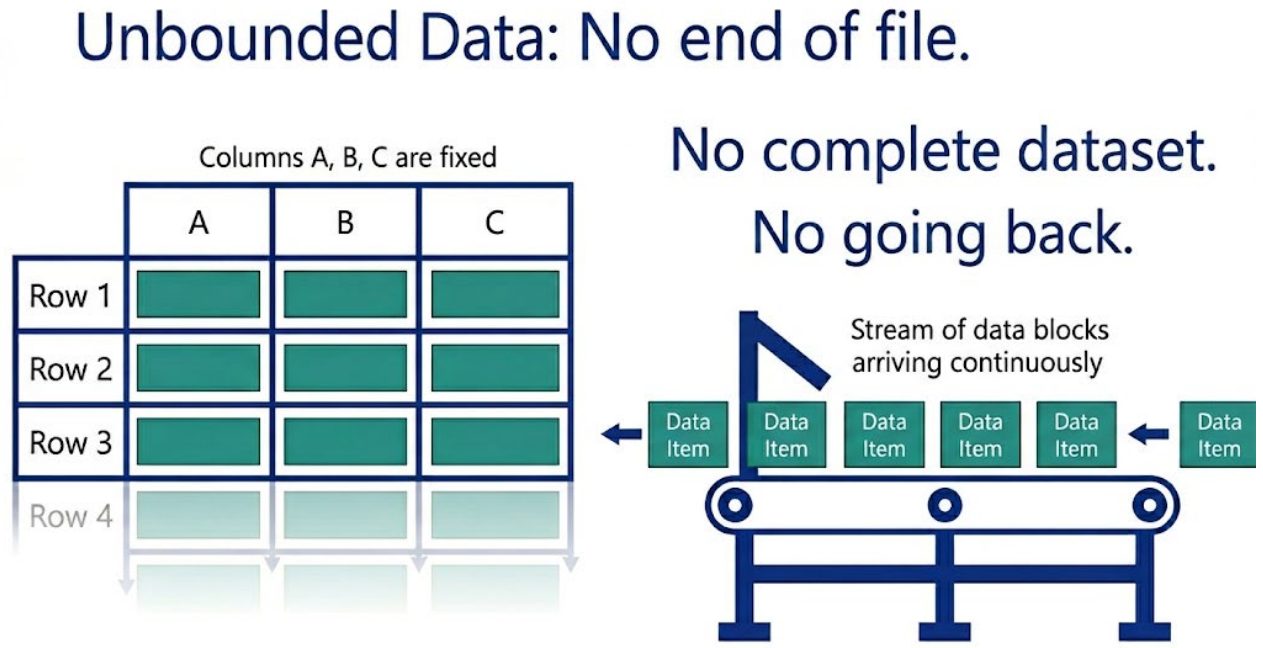
Closed Form:	Exact algebra
Gradient Descent:	Iterative optimization
SGD / Mini-Batch:	Stochastic approximation
Approximate Algorithms:	Bounded-error structures
Sampling:	Statistical subsets

Each step trades precision for feasibility. The flip: right algorithm depends on environment. The triangle: speed, accuracy, scalability -> pick two.

"Act 2 showed where the optimizer stops and you take over. Act 3 shows what happens when you push these ideas to their extreme: data that never ends."

WHAT IF THE DATA NEVER STOPS?

We just saw algorithms that process data without seeing all of it — SGD on mini-batches, HLL maintaining a sketch, reservoir sampling.



Streaming is the limit case of Act 2. Every optimization from approximate algorithms becomes mandatory, not optional.

LATENCY IS A CHOICE (DON'T OVERENGINEER)

Mode	Latency	Example Use Cases
Batch	Hours to days	Monthly reports, Census tabulations
Micro-batch	Seconds to minutes	Dashboard refresh, near-real-time
True streaming	Milliseconds	Fraud detection, 911 dispatch, clinical alerts

Decreasing latency is costly. Each step adds engineering complexity, infrastructure cost, and operational burden.

"What latency does the business actually need? If your dashboard refreshes every 15 minutes and nobody notices, you don't need sub-second streaming."

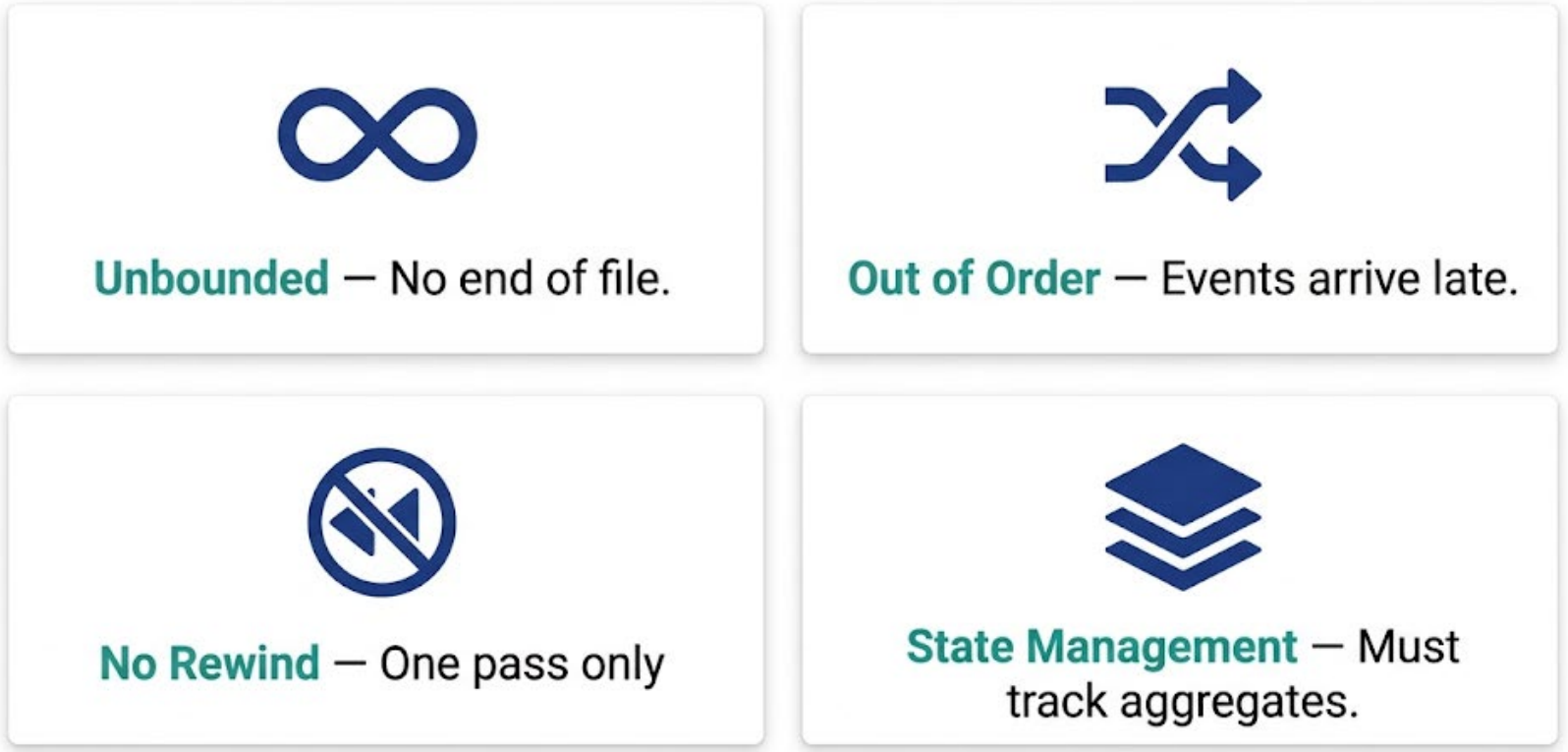
WHAT IS STREAMING? (THE SAS ANALOGY)

	SAS DATA Step	Streaming
Data source	Bounded file (known end)	Unbounded stream (no end)
Pass count	Can re-read	One pass (no rewind)
Aggregation	After complete file	Running state
Late data	Not applicable	Events arrive out of order

Because there's no end, streaming requires: windows (time boundaries), watermarks (late data), and state management (running aggregates).

Streaming = sequential processing of data that doesn't stop. The complexity comes from managing unboundedness.

WHY STREAMING IS HARD (FOUR CONSTRAINTS)

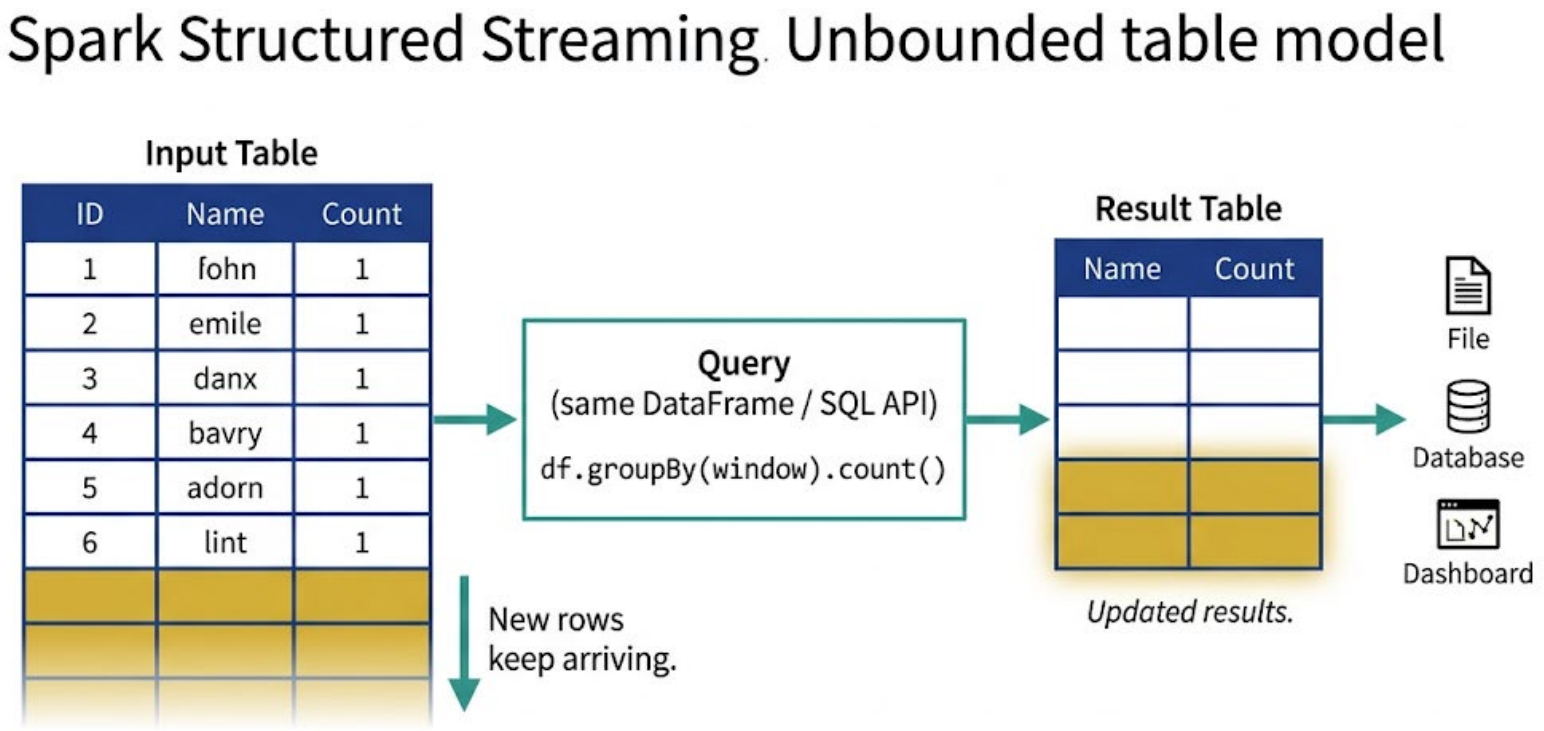


All four constraints stem from one fact: the data has no end.

All four constraints stem from one fact: the data is unbounded.

STRUCTURED STREAMING: THE UNBOUNDED TABLE

Same DataFrame API.
Same SQL.
But the table keeps growing.
Spark processes new rows as they arrive.



Same API as batch — the table just never finishes loading.

The same Spark code works for batch and streaming. The execution model changes, not the API.

WINDOWED AGGREGATIONS

Tumbling

Non-overlapping fixed blocks.
[0–5min] [5–10min] ...
Each event in exactly one window.
Simplest to reason about.

Sliding

Overlapping windows.
5-min window, 1-min slide.
Each event in multiple windows.
Smooth but more computation.

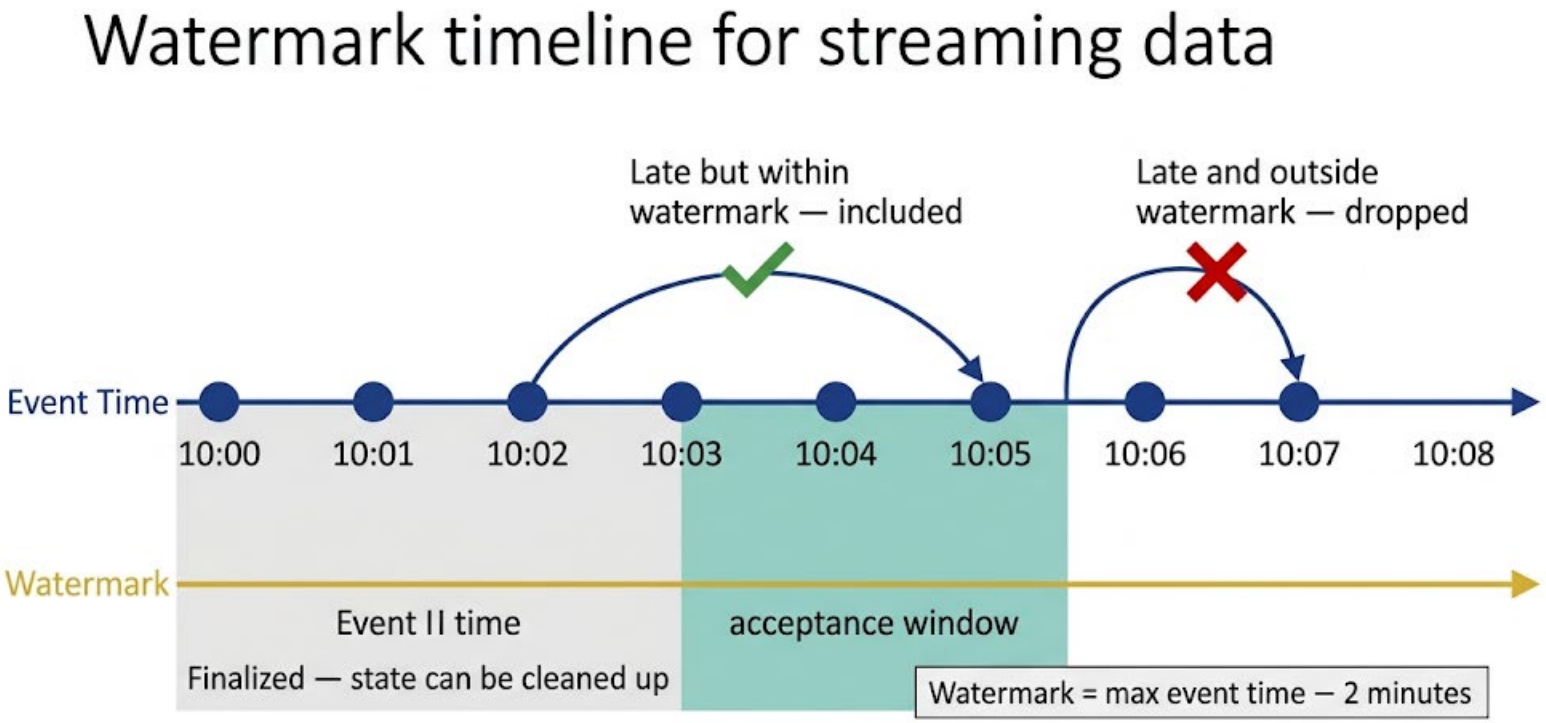
Session

Gap-based windows.
New window if gap > threshold.
Matches user behavior patterns.
Variable-length.

Windows are how streaming systems create finite boundaries in infinite data.

WATERMARKS AND LATE DATA

Watermark = "events older than this threshold are considered final."
Tradeoff: completeness vs. latency.



Watermarks formalize the tradeoff between waiting for late data and producing timely results.

STATE MANAGEMENT (SPARK 4.0)

Stateful operations require persistent, fault-tolerant, bounded state. Spark 4.0 introduces major improvements.

Arbitrary State v2

New API for custom stateful logic.
More flexible than v1.
Better error handling.

TTL for Stale State

Time-to-live for state entries.
Automatic expiration.
Prevents unbounded growth.

RocksDB Off-Heap

State stored outside JVM heap.
Avoids GC pressure.
Scales to large state sizes.

Managing this lifecycle is what makes streaming engineering hard.

Spark 4.0: automatic TTL prevents unbounded growth.

APPROXIMATE ALGORITHMS MAKE STREAMING POSSIBLE

Full circle: the approximate structures from Act 2 are requirements in streaming, not optimizations.

HyperLogLog

Distinct counts in streaming.
Fixed memory, mergeable.
No rewind needed.

Count-Min Sketch

Frequency tracking.
One-pass operation.
Bounded memory.

Reservoir Sampling

Representative samples from unbounded data.
Fixed k items.
One pass.

In batch, approximation is an optimization. In streaming, it's a requirement. You cannot store or re-read unbounded data.

STREAMING POLICY EXAMPLES

FAA	EPA	FEMA	FinCEN/SEC
			
Real-time airspace monitoring.	Wildfire smoke sensor streams.	Disaster response allocation.	Financial surveillance.

Each requires sub-minute latency, approximate algorithms, and continuous state management.

THE COUNTERPOINT: WHEN NOT TO DISTRIBUTE

DuckDB

Embedded columnar analytics.
Single machine.
Handles 50GB–1TB on modern VMs.
Benchmarks match Spark at moderate scale.

Spark

Multi-terabyte ETL.
Streaming workloads.
Massive shuffles.
Distributed joins across clusters.

2026 guideline: Spark for multi-TB ETL, streaming, massive shuffles. DuckDB for exploration and analytics under ~1TB.

"Knowing when NOT to distribute is as important as knowing how."

THE ENVIRONMENTAL COST OF COMPUTE

945 TWh

IEA projection: data center electricity by 2030 (doubled from today)

- Green AI paper: efficiency is an equity issue — who can afford to train, who gets left out?
- DuckDB: ~10× more energy-efficient than Spark on moderate workloads.
- An analyst running $O(n^2)$ on 100 nodes when $O(n \log n)$ on one would suffice is wasting electricity, water, and taxpayer money.

Algorithmic efficiency is a sustainability and equity issue, not just a performance question.

Synthesis

WHAT YOU TAKE WITH YOU

1. What's the complexity class of my algorithm?
2. Does it parallelize naturally, or is it inherently sequential?
3. What precision does this decision actually require?
4. Am I processing bounded or unbounded data?
5. Does my algorithm match my execution environment?